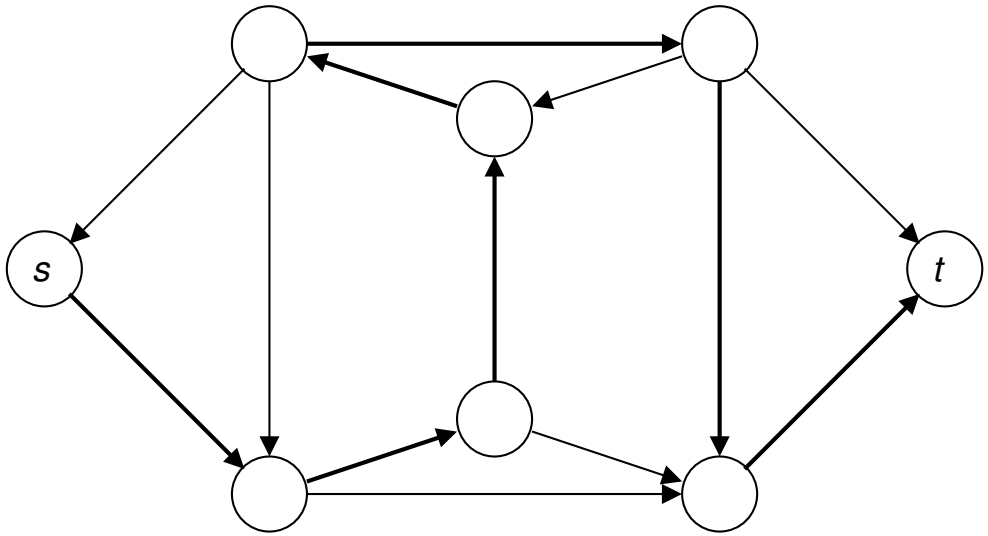
Time Complexity 03

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Hamiltonian Path

- A **Hamiltonian path** in a directed graph G is a directed path that goes through each node exactly once
- Formally,

$$HAMPATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$$



Verifier

- Suppose a path is represented by a sequence of pairs of nodes
- For example: $((s, a), (a, c), (c, d), (d, r), (r, t))$
- Given a path c, we can verify whether it is a Hamiltonian path of graph G from s to t in polynomial time
  - Given an input $\langle\langle G, s, t\rangle, c\rangle$, where G is a directed graph, s and t are nodes, and c is a path, a TM can verify whether c is a Hamiltonian path of graph G from s to t in polynomial time
- Similarly, given a natural number c, we can verify whether c is a factor of a composite number x in polynomial time
  - Given an input $\langle x, c\rangle$ where x and c are natural numbers, a TM can verify whether c is a factor of x in polynomial time
- This feature is called **polynomial verifiability**
  - Recall that both problems need exponential time to solve
  - Verifying is easier than finding

Verifier

- Recall the language $COMPOSITES$:

$$COMPOSITES = \{\langle x \rangle \mid x = pq \text{ for natural numbers } p, q > 1\}$$

- We can define the identical set using a different predicate

$$COMPOSITES = \{\langle x \rangle \mid x \text{ is divisible by } c \text{ a natural number } c > 1\}$$

- Check whether x is divisible by c can be easily done by this TM V:

  $V =$ "On input $\langle x, c \rangle$ where x and c are natural numbers greater than 1:
  1. Calculate $y = x \bmod c$
  2. If $y = 0$, *accept*. If $y \neq 0$, *reject*."
- With the TM V above, now we have

$$COMPOSITES = \{\langle x \rangle \mid V \text{ accepts } \langle x, c \rangle \text{ for some } c\}$$

- This TM V is called a **verifier**

Verifier

- A **verifier** for a language A is an algorithm V, where

$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

- In our discussion, an algorithm V is a TM
  - If TM V runs in polynomial time, it is called a **polynomial time verifier**
  - A language A is **polynomially verifiable** if it has a polynomial time verifier
- c is called a **certificate** or **proof** of membership in A
- Examples:
  - A certificate of $HAMPATH$ problem for $\langle G, s, t \rangle$ is simply a Hamiltonian path from s to t
  - A certificate of $COMPOSITES$ problem for a number x is one of its divisors

Class NP

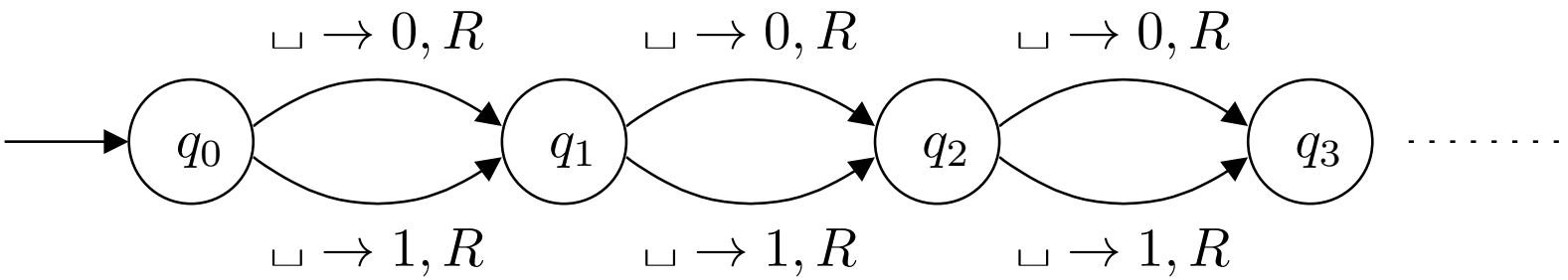
Definition 7.19

NP is the class of languages that have polynomial time verifiers.

- NP comes from **Nondeterministic Polynomial** time
- NP-Problems are set of problems in NP
- Every problems in P also have polynomial time verifier
- Therefore, $P \subseteq NP$

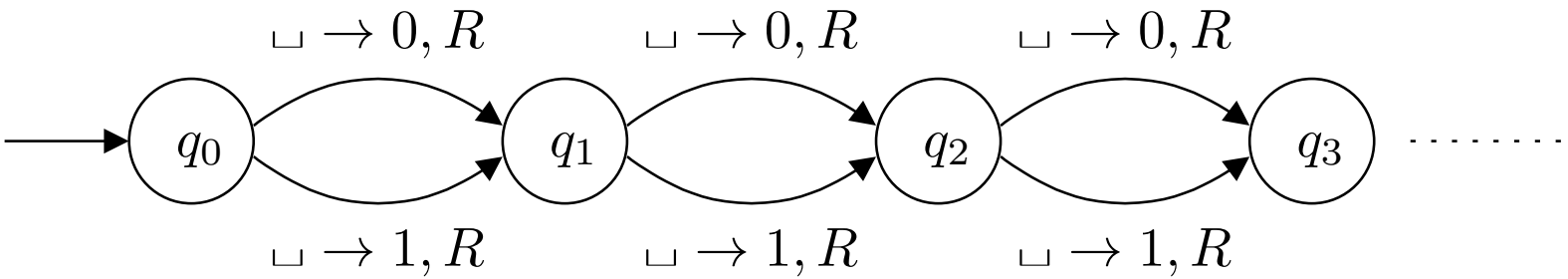
Nondeterministic

- Recall that NP stands for Nondeterministic Polynomial
- The word **nondeterministic** has been discussed int
  - Nondeterministic Finite Automaton (NFA)
  - Nondeterministic Turing Machine (NTM)
- We have not yet seen a nondeterministic Turing machine yet
- Example:



- When it read the blank symbol, it splits to two copies, same current state, but different tape contents

Nondeterministic

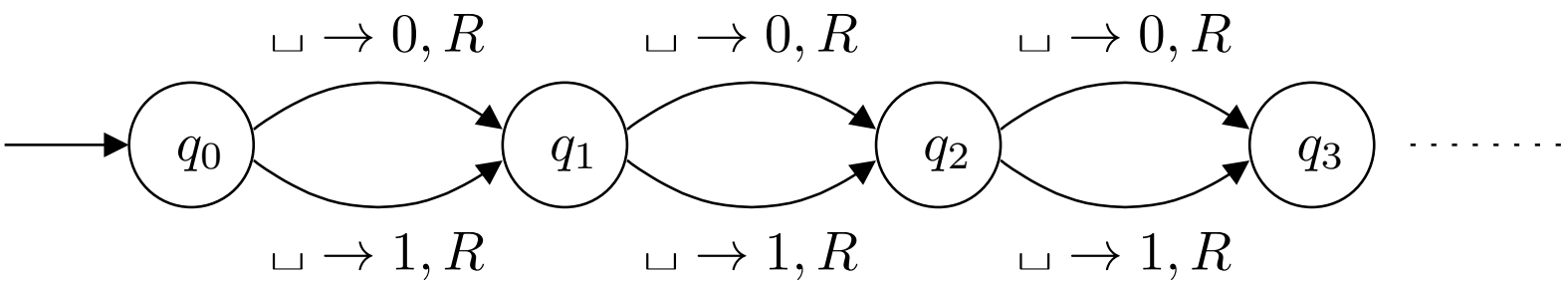


- What would happen if we run the above machine on input ε for three steps?
  - Consists of 8 copies
  - They are all in state q_3
  - Tape contents are unique:

$$000, 001, 010, 011, 100, 101, 110, \text{ and } 111$$

- This is how we generate all possible strings over $\Sigma = \{0, 1\}$ of length exactly 3

Nondeterministic



- But how to express the above TM in a high-level description?

  $M =$ "On input w:

  1. Write a string of length 3 where each symbol on the string is **nondeterministically** selected to be either 0 or 1
  2. ...

- This type of NTM is suitable for describing brute-force algorithm
  - A brute-force algorithm generally generates all possible of something
  - Then search for an element in the set of possibilities
  - If it finds an element that satisfies the criteria, done

Nondeterministic TM for $HAMPATH$

- A nondeterministic TM N_1 that decides $HAMPATH$:

  N_1 = "On input $\langle G, s, t \rangle$, where G is a directed graph with nodes s and t:

  1. Write a list of m numbers $p_1, \ldots p_m$, where m is the number of nodes in G. Each number in the list is nondeterministically selected to be between 1 and m.
  2. Check for repetitions in the list. If any are found $reject$.
  3. Check whether $s = p_1$ and $t = p_m$. If either fail, $reject$.
  4. For each i between 1 and $m - 1$, check whether (p_i, p_{i+1}) is an edge of G. If any are not, $reject$. Otherwise, all tests have been passed. so $accept$.

- The nondeterministic part of N_1 is in step 1
  - It generates all permutations of m nodes

Nondeterministic TM for $HAMPATH$

- What is the run-time of NTM N_1?
  - Step 1 takes only m steps to write m numbers
  - Check for repetition take $\mathcal{O}(m^2)$
    - Get the first number and compare with the rest is $\mathcal{O}(m)$
    - Get the second number and compare with the rest is $\mathcal{O}(m)$
    - and so on (m times)
  - Check whether the first number is s and the last number is t is $\mathcal{O}(m)$
  - Step 4 takes $\mathcal{O}(m^2)$
    - Get the first pair and scan for the edge is $\mathcal{O}(m)$
    - Get the second pair and scan for the edge is $\mathcal{O}(m)$
    - and so on ($m-1$ times)
- Note that all copies run steps 2, 3, and 4 simultaneously
- The above TM runs in polynomial time
  - From Theorem 7.11, this algorithm will run in exponential time on a single-tape Turing machine

Language in NP

- Is there a relation between problem in NP and NTM?
- Yes:

  Theorem 7.20: A language is in **NP** iff it is decided by some nondeterministic polynomial time Turing machine.
- But the NP is the class of languages that have polynomial time **verifiers**
- Proof Idea:
  - Show how to convert a polynomial time verifier to an equivalent polynomial time nondeterministic Turing machine (decider)
  - Show how to convert polynomial time nondeterministic Turing machine (decider) to an equivalent polynomial time verifier

Verifier to NTM

- Assume that $A \in NP$
  - By definition, there exists a TM V (a polynomial time verifier) for the language A that can verify an input of length n in n^k steps for some k
  - Recall that a **verifier** for a language A is an algorithm V, where

$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

- Construct a NTM N as follows:

  $N =$ "On input w of length n:

  1. Nondeterministically select string c of length at most n^k.
  2. Run V on input $\langle w, c \rangle$.
  3. If V accepts, *accept*; otherwise, *reject*."

NTM to Verifier

- Assume that A can be decided by NTM N in polynomial time
- Construct a polynomial time verifier V as follows:

  $V =$ "On input $\langle w, c \rangle$, where w and c are strings:

  1. Simulate N on input w, treating each symbol of c as a description of the nondeterministic choice to make at each step
  2. If this branch of N's computation accepts, $accept$; otherwise, $reject$."

Language in NP

Definition 7.21

$$\text{NTIME}(t(n)) = \{L \mid L \text{ is a language decided by an } \mathcal{O}(t(n)) \text{ time nondeterministic Turing machine}\}.$$

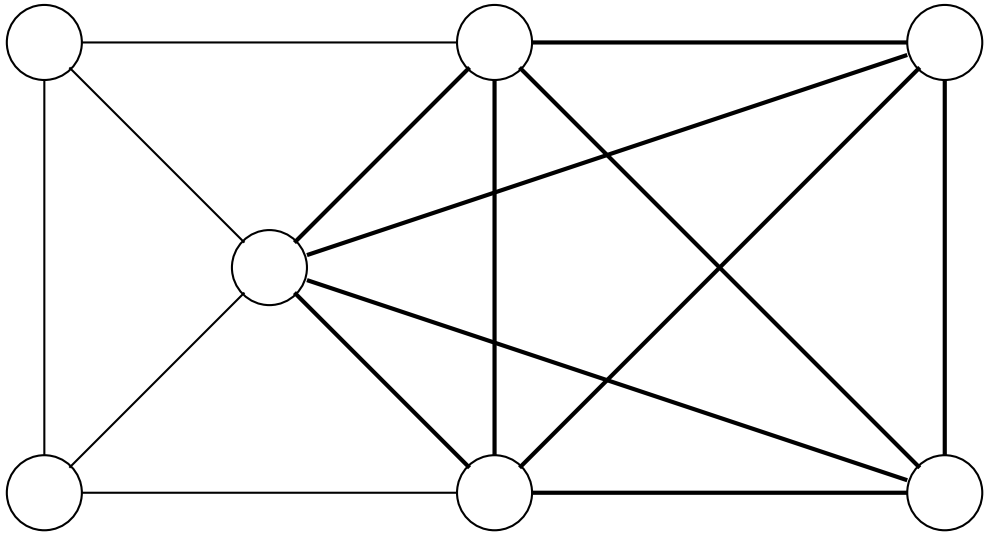
Corollary 7.22

$$\textbf{NP} = \bigcup_k \text{NTIME}(n^k)$$

- To show that a language A is in NP:
  - Construct a **nondeterministic** TM M that decides A
  - Verify that M runs in polynomial time

$CLIQUE$

- A **clique** in an undirected graph is a subgraph where every two nodes in the subgraph are connected by an edge
- A **k-clique** is a clique that contains k nodes
- A graph with 5-clique is shown below:



- Formally,

$$CLIQUE = \{\langle G, k\rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}$$

- Is $CLIQUE$ in NP?

$CLIQUE \in NP$

- To show that $CLIQUE \in NP$, we can perform one of the following:
  - Show a polynomial time verifier V for $CLIQUE$ or
  - Show an NTM N that decides $CLIQUE$ in polynomial time
- The following TM V verifies $CLIQUE$ in polynomial time:

  $V =$ "On input $\langle \langle G, k \rangle, c \rangle$:

  1. Test whether c is a subgraph with k nodes in G.
  2. Test whether G contains all edges connecting nodes in c.
  3. If both pass, $accept$; otherwise, $reject$."

$CLIQUE \in NP$

- Analysis of TM V:
  - Each step will be executed only once
  - Step 1: Need to go back and forth k times
    - Each time is $\mathcal{O}(n)$ step
    - Thus step 1 is $k \times \mathcal{O}(n) = \mathcal{O}(n)$
  - Step 2: for each node $a \in c$, it needs to find either (a, b) or (b, a) for every $b \in c$ and $b \neq a$
    - For each node, it needs to scan back and forth $k - 1$ times and each time is $\mathcal{O}(n)$
    - There are k nodes in c. Thus, step 2 is $k \times (k - 1) \times \mathcal{O}(n) = \mathcal{O}(n)$
  - Thus, V runs in polynomial time

$CLIQUE \in NP$

- The following NTM N can decide $CLIQUE$ in polynomial time:

  $N =$ "On input $\langle G, k \rangle$, where G is a graph:

  1. **Nondeterministically** select a subset c of k nodes of G.
  2. Test whether G contains all edges connecting nodes in c.
  3. If yes, $accept$; otherwise, $reject$."

- Analysis of NTM N:
  - Each step will be execute only once
  - Step 1: To generate a subset c of k nodes of G requires k steps
  - Step 2: This step is the same as Step 2 of V which is $\mathcal{O}(n)$ steps
  - Step 3: Simply $\mathcal{O}(1)$ or $\mathcal{O}(n)$ depending on implementations
  - Thus N can decides $CLIQUE$ in polynomial time

$SUBSET-SUM$

- Consider a set of numbers $S = \{x_1, x_2, \ldots, x_k\}$ and a target number t
- Is there exists a set $Y = \{y_1, y_2, \ldots, y_i\} \subseteq S$ such that $\sum_i y_i = t$?
- Formally:

$$SUBSET-SUM = \{\langle S, t\rangle \mid S = \{x_1, x_2, \ldots, x_k\}, \text{ and for some } \{y_1, y_2, \ldots, y_i\} \subseteq \{x_1, x_2, \ldots, x_k\}, \text{ we have } \sum_i y_i = t\}$$

- For example $\langle\{4, 11, 16, 21, 27\}, 25\rangle \in SUBSET-SUM$ because $4 + 11 = 25$
- Note that for $SUBSET-SUM$, S and Y are multisets which allow duplicate elements

$SUBSET-SUM \in NP$

- To show that $SUBSET-SUM \in NP$, we can perform one of the following:
  1. Show a polynomial time verifier V for $SUBSET-SUM$
  2. Show an NTM N that decide $SUBSET-SUM$ in polynomial time
- The following TM V verifies $SUBSET-SUM$ in polynomial time:

  $V =$ "On input $\langle \langle S, t \rangle, c \rangle$:
  1. Test whether c is a collection of numbers that sum to t.
  2. Test whether S contains all the numbers in c.
  3. If both pass, $accept$; otherwise, $reject$."
- The following NTM N decides $SUBSET-SUM$ in polynomial time:

  $N =$ "On input $\langle S, t \rangle$:
  1. Nondeterministically select a subset c of the number in S.
  2. Test whether c is a collection of numbers that sum to t.
  3. If the test passes, $accept$; otherwise, $reject$."